

Sức mạnh của thống kê: tiếp xúc toàn diện với cây đoạn

Zhang Kunwei, Đại học Thanh Hoa

1 Từ thống kê đến cây đoạn

Theo cách phân loại của D. E. Knuth, thuật toán máy tính có thể chia thành hai loại lớn: thuật toán số và thuật toán phi số. Trong nhóm phi số có các thao tác như lập chỉ mục, phân loại, thống kê, v.v. Cây đoạn là một công cụ rất tự nhiên cho lớp bài toán thống kê trên khoảng.

Nhưng khi nhắc đến cây đoạn, người ta thường than phiền:

- hằng số lớn;
- khó viết;
- khó gỡ lỗi;
- khó nghĩ ra.

Có một bài trên POJ với giới hạn thời gian rất chặt. Hầu hết mọi người dùng cây Fenwick, nhưng nếu chỉ biết cây đoạn thì sao? Hơn nữa, ta hoàn toàn có thể sửa bài thành một phiên bản không dùng được cây Fenwick. Sau nhiều lần TLE với cây đoạn, có người đăng bài than: “Lần sau viết cây đoạn không đệ quy bằng hợp ngữ, còn quá thời gian nữa không?” Nhưng đoạn mã đã quá thời gian ấy dài tới khoảng 2 KB.

Thực ra lời giải của tác giả dùng cây đoạn: chạy rất nhanh và mã dưới 1 KB. Điểm mấu chốt không phải là cây đoạn tự nó nặng, mà là phải cài đặt linh hoạt. Cây đoạn có thể:

- chạy nhanh;
- thích nghi tốt;
- viết thuận tiện;
- có cấu trúc đơn giản;
- dễ gỡ lỗi.

Vậy vì sao trong *Introduction to Algorithms* và trong “sách đen” lại khó thấy bóng dáng của nó?

Trong quá trình phát triển lâu dài, hình học tính toán sinh ra nhiều cấu trúc dữ liệu thực dụng. Truy vấn khoảng và truy vấn đâm xuyên đều là các vấn đề được hình học tính toán xử lý. Xem như một trường hợp đặc biệt của các trường hợp đặc biệt, cây đoạn giải bài toán:

thống kê hình học trên không gian một chiều.

Các mở rộng lên chiều cao hơn, chẳng hạn kd-tree và những cấu trúc tương tự, có thể thực hiện truy vấn trực giao. Vì nội dung hình học tính toán trong thi lập trình có giới hạn, phần này không được mở rộng thêm.

Cây đoạn chia trục số thành các khoảng để xử lý, ví dụ $[8, 10)$, $[10, 11)$, v.v. Nhưng trong thực tế ta thường đối mặt với đại lượng rời rạc. Vì vậy cây đoạn trong thi đấu thường thoái hóa thành một “cây điểm”: đoạn ở tầng đáy chỉ chứa một điểm. Chẳng hạn $[8, 9)$ chỉ chứa điểm nguyên 8. Trong các phần sau, ta không phân biệt “cây điểm” và cây đoạn.

2 Bài toán kinh điển: tổng đoạn

Hãy xét truy vấn $[1, 4)$ trong miền $[0, 4)$. Cây đoạn tách miền thành $[0, 2)$ và $[2, 4)$; nhánh trái lại tách thành hai lá 0 và 1. Khi hỏi $[1, 4)$, ta lấy lá 1 và cả đoạn $[2, 4)$.

Tuy mỗi lần tưởng như có thể phải đệ quy vào cả hai cây con, trên thực tế mỗi tầng chỉ thăm nhiều nhất hai nút. Lý do là khoảng cần hỏi liên tục. Nếu trong cùng một tầng có ba nút được thăm, gọi theo thứ tự là A, B, C , thì do truy vấn liên tục, đã có A và C thì nhất định cũng bao gồm anh em của B trong cây. Khi đó ta có thể dùng trực tiếp cha của B để tính, không cần dùng B .

Nói thực dụng một chút: thứ không có ích thì không học.

Với riêng tổng đoạn, ta có thể xử lý mảng ban đầu thành tổng tiền tố:

```
for i = 2 to n do
    A[i] += A[i-1]
```

```
Ans(a, b) = A[a] - A[b-1]
```

Một bài tổng đoạn nhỏ bé, có cần dùng cây đoạn không? Lý do là tổng đoạn có tính chất rất đẹp: nó thỏa **phép trừ khoảng**. Phép trừ khoảng sẽ còn được nhắc lại sau. Tuy nhiên tổng tiền tố trực tiếp không phải vạn năng: nếu phải sửa phần tử thì sao?

Cấu trúc dữ liệu	Sửa phần tử	Lấy tổng tiền tố
Lưu trực tiếp mảng gốc	$O(1)$	$O(n)$
Lưu trực tiếp tổng tiền tố	$O(n)$	$O(1)$
Cây đoạn	$O(\log n)$	$O(\log n)$

Vấn đề này vốn tùy quan điểm mỗi người. Nhưng phần sau nhấn mạnh một cách cài đặt không hoàn toàn “vốn dĩ” như vậy.

Một cách truy cập cây đoạn thường thấy là:

- nếu cần cả đoạn hiện tại thì trả về ngay;
- nếu giao với đoạn con trái thì đệ quy xử lý;
- nếu giao với đoạn con phải thì đệ quy xử lý;
- hợp nhất các kết quả thu được.

Đáng tiếc, phương pháp ngây thơ này không dễ cài đặt đến thế, và còn có vấn đề hiệu năng nghiêm trọng: hằng số quá lớn.

3 Lưu trữ kiểu heap

Muốn làm tốt việc gì, trước hết phải mài sắc công cụ.

Ta bắt đầu bằng cây nhị phân đầy đủ. Dù có thể thiết kế cây đoạn ba nhánh, bốn nhánh, v.v., ở đây cứ bắt đầu từ cây nhị phân. Cây cao thì đường đi dài, nhưng nếu cây không đủ nhánh thì lại phải xét nhiều trường hợp; cây nhiều nhánh để sau hãy bàn.

Để tránh xử lý nhiều tình huống đặc biệt, giả sử cây nhị phân là đầy đủ. Nếu tổng miền là $[0, 1000)$, cứ xem như $[0, 1024)$ là đủ.

Lưu trữ kiểu heap là then chốt:

con trái của N là $2N$, con phải của N là $2N + 1$, cha của N là $N \gg 1$.

Chẳng hạn cây có gốc 1, hai con 2, 3, bốn lá 4, 5, 6, 7. Nếu viết chỉ số dưới dạng nhị phân, bốn lá là 100, 101, 110, 111. Đây cũng là một cây chữ cái: mỗi nút lưu tổng của mọi nút có nhãn bắt đầu bằng tiền tố ấy. Ví dụ nút 1 lưu tổng của cả bốn lá có tiền tố 1. Từ 100 đến 111 vừa đúng là mảng gốc; đường như chỉ số lá trừ 4 là chỉ số mảng.

Vì thế ta có thể tìm trực tiếp lá tương ứng với một số, không cần đi từ trên xuống tìm từng bước. Nếu đáy bắt đầu ở M , thì lá của vị trí x là $x + M$.

4 Truy vấn không đệ quy

Với truy vấn đóng $[s, t]$, ta đổi thành khoảng mở $(s - 1, t + 1)$, rồi đi từ hai lá tương ứng lên trên. Khi hai điểm biên chưa là hai anh em kề nhau, ta xét những đoạn hoàn toàn nằm giữa chúng. Ba dòng đầu của mã dưới lần lượt đổi đoạn đóng thành đoạn mở và đưa hai đầu nút tới vị trí lá.

```
Func Query(s, t)
    s = s - 1, t = t + 1
    s += M, t += M
    While not ((s xor t) == 1) do
        If ((s and 1) == 0) Answer += Tree[s + 1]
        If ((t and 1) == 1) Answer += Tree[t - 1]
        s = s >> 1, t = t >> 1
```

Viết gọn trong C:

```
for (s=s+M-1, t=t+M+1; s^t^1; s>>=1, t>>=1) {
    if (~s&1) Ans+=T[s^1];
    if ( t&1) Ans+=T[t^1];
}
```

Cần nhớ một chi tiết: khi đổi khoảng đóng thành khoảng mở, chỉ số có thể vượt ra ngoài một đơn vị. Về lý thuyết cây chứa được $[0, 2^N)$, nhưng cách hỏi này thực ra chỉ hỏi được miền $[1, 2^N - 2]$. Nếu cần hỏi vị trí 0, hãy dịch toàn bộ chỉ số về sau một đơn vị, tức mọi chỉ số cộng thêm 1. Nếu cần hỏi đoạn kết thúc ở 1023 trong miền $[0, 1024)$, mà dữ liệu thật chỉ cỡ 1000, thì có thể tăng tổng miền thành $[0, 2048)$.

Các hình minh họa trong slide cho truy vấn $(0, 5)$ trên cây có lá 8 đến 15 cho thấy cách đi tự nhiên từ hai biên lên trên:

- biên trái xuất phát trước lá 8, biên phải xuất phát sau lá 13;
- các đoạn hoàn toàn nằm giữa hai biên được cộng vào đáp án, ví dụ các nút đại diện cho phần giữa;
- sau mỗi bước hai biên cùng đi lên cha, nên chỉ chạm một số nút tuyến tính theo chiều cao cây.

5 Sửa điểm và dựng cây

Sửa một phần tử cũng đi từ lá lên gốc:

```
Func Change(n, NewValue)
    n += M
    Tree[n] = NewValue
    While n > 1 do
```

```
n = n >> 1
Tree[n] = Tree[2n] + Tree[2n+1]
```

Viết gọn:

```
for(T[n+=M]=V, n>=>1;n>=>1)
    T[n]=T[n+n]+T[n+n+1];
```

Hết chưa? Hết rồi.

Cách này chỉ dùng gấp đôi không gian mảng gốc, trong đó còn chứa nguyên vẹn mảng gốc ở tầng lá. Việc dựng cây rất dễ:

```
For i = M-1 downto 1 do
    T[i] = T[2i] + T[2i+1]
```

Nó dễ viết, dễ hiểu, đi từ dưới lên, mỗi nút chỉ thăm một lần, và không nhất thiết phải thăm tới tầng trên cùng. Trong thực tế nó rất nhanh, gần với cây Fenwick.

Nếu chỉ hỏi tổng tiền tố, không hỏi tổng đoạn, thì do cây Fenwick dựa vào phép trừ khoảng, còn ta chỉ cần tiền tố, cây đoạn cũng có thể dùng một lần không gian. Các slide minh họa cho thấy khi chỉ giữ những nút cần thiết cho các tiền tố, tất cả con phải không còn dùng đến. Ta có một cây nơi mỗi nút ghi dạng “chỉ số–giá trị”, chẳng hạn 1 không lưu gì, 2 lưu phần của nút 1, 4 lưu phần của nút 2, 6 lưu phần của nút 3, còn mọi con phải bị bỏ.

Nhìn lại, đây chẳng phải giống cây Fenwick sao? Vốn dĩ nó nên giống như vậy. Cây Fenwick chính là cây đoạn sau khi bỏ một nửa không gian và dùng thứ tự trung tự; khi tính vị trí cần dùng `lowbit`. Cách ở đây dùng thứ tự tiền tự, phù hợp hơn với cách nghĩ trực quan.

Tác giả từng dùng kiểu viết này làm nhiều bài. Người khác nói mã khó hiểu. Tác giả bảo đó là cây Fenwick; người viết cây Fenwick nói không có `lowbit`. Tác giả bảo vậy cứ xem là cây đoạn; người khác lại không tin cây đoạn không đệ quy có thể ngắn như thế.

6 Lười biếng là đức hạnh

Bắt đầu ác mộng: RMQ, tức **Range Minimum Query**, truy vấn giá trị nhỏ nhất trên đoạn. Cây đoạn có thể hỗ trợ sửa đoạn:

toàn bộ giá trị trong một đoạn cùng tăng một số, có thể âm hoặc dương.

Sửa thô bạo không còn hiệu quả.

Cách thường gặp là thêm một nhãn ở mỗi nút, biểu thị đoạn này từng được tăng bao nhiêu. Trong quá trình đệ quy từ trên xuống, nếu thấy nhãn thì cập nhật giá trị nút hiện tại và đẩy nhãn xuống con. Nhưng như vậy lại phải đi từ trên xuống.

Lưu trữ kiểu heap cũng có thể truy cập từ trên xuống; cùng lắm đi lên và đi xuống mỗi chiều một lần. Tuy nhiên có cách tốt hơn: không đẩy nhãn xuống, mà khi cần thì tra. Trong truy vấn từ dưới lên, mỗi lần đi lên một tầng ta dùng nhãn tương ứng để hiệu chỉnh đáp án. Ta muốn giữ càng nhiều thông tin càng tốt ở gần gốc, vậy cần gì phải đẩy nhãn xuống?

6.1 Nhãn vĩnh viễn

Xét một đoạn thẳng hỗ trợ tô màu đoạn. Hỏi một đoạn có đồng màu hay không. Nhãn màu có thể là:

1 = đỏ, 2 = xanh lam, 3 = xanh lục, 0 = lẫn màu.

Khi hỏi, ta có thể nhìn trực tiếp vào nhãn.

Nhưng nếu nút 2 là đỏ, nút 3 là xanh lam, nút 1 là lần màu, sau đó sửa nút 3 thành đỏ, hỏi nút 1 thì còn lần màu không? Nhãn vĩnh viễn chính là giá trị, và giá trị được tự động duy trì. Thuật toán sửa khi đổi nút 3 đã phải duy trì nút 1: trên đường đi từ dưới lên, tính lại nhãn của mọi nút gặp được:

```
If (Mark[x]==0 and Mark[2x]==Mark[2x+1])
    Mark[x]=Mark[2x]
```

Quay lại RMQ có sửa đoạn. Nói nôm na, nhãn là một lượng tương đối. Đã có nhãn rồi thì giá trị còn dùng để làm gì? Hay nói cách khác: nếu bản thân giá trị đã là tương đối, còn cần nhãn không? Nếu mọi số đều bắt đầu biến đổi từ 0, thì sau khi vĩnh viễn hóa nhãn, mọi giá trị tuyệt đối đều luôn bằng 0. Vì thế ta thậm chí không lưu giá trị nữa: nhãn vĩnh viễn chính là giá trị.

Mỗi nút không lưu trực tiếp giá trị lớn nhất hay nhỏ nhất $T[n]$, mà lưu

$$M[n] = T[n] - T[n \gg 1].$$

Tức là mỗi giá trị nút được trừ đi giá trị của cha. Sửa đoạn thì sửa trực tiếp $M[n]$. Truy vấn từ một nút lên gốc là cộng dồn:

$$T[n] = M[n] + M[n \gg 1] + \dots + M[1].$$

Khi gặp nút x , đặt

$$A = \min(M[2x], M[2x + 1]),$$

rồi chuẩn hóa:

$$M[x] += A, \quad M[2x] -= A, \quad M[2x + 1] -= A.$$

Mã sửa đoạn cộng x :

```
Func Add_x(s,t,x)
    for (s=s+M-1,t=t+M+1;s^t^1;s>>=1,t>>=1) {
        if (~s&1) T[s^1]+=x;
        if ( t&1) T[t^1]+=x;
        A=min(T[s],T[s^1]),T[s]-=A,T[s^1]-=A,T[s>>1]+=A;
        A=min(T[t],T[t^1]),T[t]-=A,T[t^1]-=A,T[t>>1]+=A;
    }
```

Mã truy vấn cực đại trên đoạn:

```
Func Max(s,t)
    for (s=s+M-1,t=t+M+1;s^t^1;s>>=1,t>>=1) {
        LAns+=T[s],RAns+=T[t];
        if (~s&1) LAns=max(LAns,T[s^1]);
        if ( t&1) RAns=max(RAns,T[t^1]);
    }
    Ans = max(LAns,RAns);
    while (s>1) Ans += T[s>>=1];
```

Hiệu sai là phương tiện quan trọng để biến tuyệt đối thành tương đối. Sau khi vĩnh viễn hóa, nhãn chính là giá trị, chỉ khác là giá trị này mang tính tương đối. Khi tính đáp án, ta có thể tận dụng thông tin trên đường từ nút lên gốc.

Vào thời điểm làm slide, đa số cách viết cây đoạn là từ trên xuống; trong đó nhãn thường không vĩnh viễn. Với RMQ cần đẩy nhãn xuống; với bài tô màu cần đẩy và thu thập nhãn. Ý tưởng gần với phương pháp ở đây, nhưng cài đặt khác nhiều. Ít nhất nó phải đi cả lên lẫn xuống, nên chậm hơn.

7 Hiệu sai và các phép toán đoạn

Cây đoạn là cây cầu nối mảng gốc và tổng tiền tố. Nếu lấy hiệu sai ở cả hai bên cầu, vì tổng tiền tố và hiệu sai là hai phép ngược nhau, cây đoạn cũng là cầu nối hiệu sai và mảng gốc.

Nếu cần hỗ trợ sửa đoạn và hỏi điểm, không cần nhãn: lấy hiệu sai của mảng gốc rồi dùng cây đoạn duy trì tổng tiền tố của mảng hiệu sai.

Không nhờ nhãn mà vẫn hỗ trợ sửa đoạn và tính tổng đoạn: trước hết lấy hiệu sai, bài toán biến thành duy trì “tổng tiền tố của tổng tiền tố”. Đừng bị dọa bởi tên gọi; tổng tiền tố của tổng tiền tố là:

$$\begin{aligned}SS_1 &= S_1 = x_1, \\SS_2 &= S_1 + S_2 = 2x_1 + x_2, \\SS_3 &= S_1 + S_2 + S_3 = 3x_1 + 2x_2 + x_3 \\&= 4(x_1 + x_2 + x_3) - (1x_1 + 2x_2 + 3x_3).\end{aligned}$$

Vì vậy chỉ cần duy trì thêm một tổng tiền tố có trọng số $i \cdot x_i$.

8 Dãy khoảng tăng dài nhất

Xét bài toán “dãy khoảng tăng” dài nhất: trong một dãy các khoảng, chọn được nhiều khoảng nhất theo thứ tự sao cho chúng không chồng nhau và tăng đơn điệu. Ví dụ với $[1, 3]$, $[2, 4]$, $[4, 5]$, đáp án là $[1, 3] + [4, 5]$.

Quy hoạch động có quyết định khả thi nào? Để độ dài dãy tăng lớn hơn x , số cuối cùng nhỏ nhất có thể là bao nhiêu; gọi là $f[x]$. Ta cần duy trì $f[x]$ với:

- hỏi điểm: hỏi giá trị $f[x]$;
- sửa đoạn: mọi giá trị bên trái x vượt quá K đều đổi thành K .

Đổi trái và phải một chút: toàn bộ $f[-x]$ là đơn điệu giảm. Một dãy đơn điệu giảm có thể xem là kết quả lấy tiền tố nhỏ nhất của một dãy bình thường. Phép ngược của tiền tố nhỏ nhất là gì? Ta không quan tâm.

Điều cần duy trì là dãy gốc sau phép ngược của tiền tố nhỏ nhất. Dù ta thậm chí không biết phép ngược ấy là gì cũng không sao, vì không dùng đến nó:

- hỏi điểm x : trả trực tiếp tiền tố nhỏ nhất của dãy đang duy trì;
- sửa đoạn: mọi giá trị bên trái x vượt quá K đổi thành K ; trong dãy đang duy trì, chỉ cần đổi $f[x]$ thành K .

9 Khi nào dùng được cây đoạn

Nói nhiều như vậy, chìa khóa để giải được bài bằng cây đoạn là **phép cộng khoảng**: “tính chất” của một khoảng phải do các khoảng con quyết định hoàn toàn. Tính chất ấy bao gồm nhưng không giới hạn ở tổng, cực trị, trạng thái màu. Nó hơi giống cách biểu diễn trạng thái trong quy hoạch động. Có lúc yêu cầu nhiều thông tin hơn lại khiến bài dễ hơn. Ví dụ đơn giản nhất là hỏi giá trị lớn thứ hai của đoạn. Nếu không thỏa phép cộng khoảng, mọi thứ sẽ hỏng.

Ta đều biết dùng cây đoạn để hỏi trung bình đoạn không khó. Nhưng thử hỏi trung vị của một đoạn? Nếu vẫn nói không khó, thử hỏi tiếp một của một đoạn.

Nguyên nhân bài ngày càng khó rất đơn giản: biết trung vị của hai đoạn thì có biết trung vị của hợp hai đoạn không? Biết một của từng đoạn thì dùng được gì?

9.1 Bài toán k -th number

Cho một dãy số, nhiều lần hỏi số lớn thứ k trong một đoạn. Yêu cầu nhiều thông tin hơn lại dễ hơn: mỗi nút cây đoạn phải giữ nhiều thông tin hơn. Ở mỗi khoảng, lưu toàn bộ các số trong khoảng đó thành một mảng đã sắp xếp; phép cộng khoảng được thực hiện bằng trộn.

Nếu mỗi lần truy vấn lại trộn các đoạn liên quan, độ phức tạp đạt tới $O(n)$. Thay vào đó:

- rời rạc hóa;
- nhị phân đáp án;
- biến bài toán thành: x là số lớn thứ mấy trong đoạn?

Với câu hỏi sau, ta tìm nhị phân riêng trên các mảng đã sắp của vài nút cây đoạn. Tổng độ phức tạp:

$$O(n \log n + Q \log^2 n).$$

Nếu có phép trừ khoảng, cây đoạn càng mạnh hơn nhiều. Ví dụ “tổng đoạn” biến thành “tổng tiền tố”, thao tác đơn giản hơn đáng kể. Đây cũng là điều kiện để dùng cây Fenwick. Nhưng nó không bắt buộc; hãy so sánh với phép cộng khoảng ở trên.

10 Từ băm phân đoạn đến cây đoạn động

Ta cần duy trì một cấu trúc dữ liệu hỗ trợ:

- chèn số nguyên;
- lấy giá trị lớn nhất;
- giả sử phạm vi số nguyên là 0 đến 65535.

Heap tất nhiên làm được. Nhưng trong sách đen của thầy Liu Rujia có một chiêu: “băm phân đoạn”. Chia miền thành nhiều đoạn và lưu thông tin “trong đoạn có số hay không”.

Nếu thêm nhiều tầng thì sao? Ba tầng, bốn tầng, ..., cho tới khi dưới mỗi nút chỉ còn hai điểm. Ta thu được một cây đoạn. Ví dụ băm phân đoạn có nút $[0, 65536)$, dưới nó là $[0, 256)$ và các đoạn còn lại, rồi xuống các điểm như $0, \dots, 255$. Khi chia tới các khoảng nhị phân như $[0, 4)$, $[0, 2)$, $[2, 4)$ và các điểm $0, 1, 2, 3$, đó cũng chính là cây đoạn. Đừng tự làm phức tạp quá mức.

11 Thống kê thứ tự trên miền số nguyên

Bây giờ duy trì một cấu trúc dữ liệu hỗ trợ:

- chèn số nguyên;
- xóa số nguyên;
- lấy số lớn thứ k (lấy lớn nhất, nhỏ nhất là trường hợp đơn giản);
- hỏi hạng của một số;
- hỏi một số có tồn tại không;
- cho phép phần tử trùng nhau;
- phạm vi số nguyên là 0 đến 65535.

Cây cân bằng ư? Dùng cây đoạn. Thống kê tần suất xuất hiện của mỗi số trong $[0, 65536)$, ký hiệu là $f[x]$:

- chèn số nguyên: $f[x]++$;
- xóa số nguyên: $f[x]--$;
- hỏi hạng của số: lấy tổng tiền tố;
- lấy số lớn thứ k : biết tổng tiền tố cần tìm, đi từ trên xuống; nếu phía trái không đủ thì đi sang phải, ngược lại đi sang trái.

Thực nghiệm cho thấy cây đoạn xử lý loại bài này rất nhanh. Cây cân bằng nhanh nhất trong thử nghiệm là Size Balanced Tree cần hơn 4 giây; cây đoạn giải xong dưới nửa giây, thậm chí chưa trừ thời gian đọc dữ liệu. Cây đoạn dùng chính cách cài đặt đã nói ở trên: mã cực ngắn và gỡ lỗi rất đơn giản.

Nếu miền giá trị lớn, hãy rời rạc hóa. Nếu không thể rời rạc hóa, còn có cách khác.

12 Cây chữ cái và cây đoạn

Với xâu, ta cần duy trì:

- chèn xâu;
- xóa xâu;
- lấy xâu lớn thứ k ;
- hỏi hạng của xâu;
- hỏi một xâu có tồn tại không.

Trường size trong cây chữ cái được duy trì giống hệt cây đoạn. Cách tính hạng cũng hoàn toàn giống cây đoạn trên số nguyên. Nếu xem xâu như số hệ 26, thì cây chữ cái chẳng phải là cây đoạn sao?

Trong cây chữ cái, các con được lưu bằng con trỏ, không gian cấp phát theo nhu cầu. Cây không phải cây nhị phân đầy đủ, thậm chí không hoàn chỉnh; mọi vấn đề được xử lý từ trên xuống. Cây đoạn cũng có thể làm như vậy. Miền giá trị dù lớn đến đâu, có lớn hơn 26^N không?

Với số nguyên, đó chỉ là cây đoạn cao 32 tầng, lưu bằng con trỏ, nút được sinh động như trong cây chữ cái. Nó hỗ trợ chèn, xóa, chọn phần tử, v.v. Độ phức tạp là:

$$O(N \log M),$$

trong đó M là giá trị lớn nhất. Với long int, số tầng là 32. Thử nghiệm mất hơn 1 giây, vẫn nhanh hơn cây cân bằng hơn 4 giây. Cách này đi từ trên xuống, chỉ cần tính tổng tiền tố, nên cũng không khó viết.

Một số biến thể:

- Nén giống cây chữ cái nén sẽ tiết kiệm rất nhiều không gian, nhưng khó viết hơn.
- Sau khi xóa một số, thử thu hồi những nút đã cấp phát; đổi lại hơi chậm hơn và khó viết hơn.
- Động hóa chiều cao cây: ban đầu cây cao 1, chỉ chứa được 0; mỗi lần cần chứa số quá lớn thì thêm một gốc mới, con trái là gốc cũ, con phải rỗng. Cách này khá thực dụng.

13 Cây cân bằng là cây đoạn biết xoay

Nhiều thông tin trên cây cân bằng có thể được duy trì giống cây đoạn. Cây cân bằng là một cây đoạn động biết xoay. Ví dụ đơn giản nhất là trường `size`.

Một bài làm đau lòng danh sách liên kết chia khối có chương trình chuẩn dài hơn 5 KB: duy trì một dãy số, hỗ trợ:

- tăng một số trên đoạn;
- xóa đoạn;
- chèn đoạn;
- tính tổng đoạn;
- đảo đoạn.

Cây cân bằng splay hỗ trợ xóa đoạn và chèn đoạn. Cây đoạn hỗ trợ tăng đoạn và tính tổng đoạn. Hãy đặt các giá trị kiểu cây đoạn lên các nút của cây cân bằng. Mỗi nút biểu diễn thông tin tổng hợp của chính nó và cây con. Khi cây cân bằng xoay, cập nhật các trường kiểu cây đoạn. Đó là một cây đoạn biết chuyển động.

Nếu vừa cần sửa đoạn vừa cần tổng đoạn, dùng đẩy nhãn từ trên xuống. Để xử lý đảo đoạn, thêm một biến `bool` cho biết cây con trái và phải của nút hiện tại đã bị hoán đổi. Trước hết splay đoạn cần xử lý ra khỏi cây rồi thao tác. Có cần đồng thời đổi nhãn đảo của mọi nút không? Không ghi trực tiếp nhãn đảo, mà ghi nhãn của nhãn đảo.

Tất cả các phần đều tương đối đơn giản. Viết từng chút một, nó chỉ là rất nhiều bài nhỏ ghép lại.

14 Một hướng khác cho K -th number

Về cây đoạn, ta đã nói quá nhiều. Nhưng vẫn còn một phương pháp khác cho bài K -th number. Nếu các khoảng truy vấn không bao hàm nhau, hãy sắp xếp tất cả khoảng cần hỏi để tính. Dùng cây cân bằng hoặc cây đoạn lưu các số trong khoảng hiện tại, rồi chuyển sang khoảng tiếp theo. Đầu trái tăng nghĩa là xóa số; đầu phải tăng nghĩa là thêm số. Mỗi số vào ra mỗi bên một lần.

Điểm mấu chốt là chọn cách tính hợp lý để khác biệt giữa hai khoảng liên tiếp càng nhỏ càng tốt. Chi phí chuyển từ một khoảng sang khoảng khác là gì? Xem mỗi khoảng như một điểm trên mặt phẳng, với tọa độ là hai đầu mút; chi phí là khoảng cách Manhattan giữa hai điểm. Rồi sao nữa, tìm đường Hamilton? Không: một khoảng đã biết có thể dùng để tính nhiều khoảng chưa biết. Hãy dùng cây khung nhỏ nhất theo khoảng cách Manhattan trên mặt phẳng.

Cây khung nhỏ nhất Manhattan của đồ thị phẳng có thể tìm trong $O(n \log n)$. Trước hết xử lý Q câu hỏi bằng phương pháp này, rồi tính thực tế theo thứ tự và cách đi trên MST. Phân tích tổng độ phức tạp được để lại cho người giỏi toán. Tuy phải vòng vèo khá nhiều, nhưng đây là cảm giác dùng mô hình để giải quyết vấn đề thực tế: hóa ra MST cũng có thể dùng làm tiền xử lý.

15 Bài luyện tập

Nghe lâu như vậy, hãy cùng làm một bài luyện tập:

Cho một dãy n số và m khoảng. Với mỗi khoảng, hãy tìm số lần xuất hiện của một trong khoảng đó. Ràng buộc:

- với 10% dữ liệu: $n < 100, m < 100$;
- với 30% dữ liệu: $n < 1000, m < 1000$;
- với 50% dữ liệu: $n < 100000, m < 100000$;
- với 70% dữ liệu: $n < 1000000, m < 1000000$; trong các dữ liệu trên, các khoảng không bao hàm nhau;
- với 30% còn lại: $n < 10000, m < 10000$, các khoảng có thể bao hàm nhau.

Liên hệ trong slide gốc: aceeca.135531@gmail.com.